

Case Study Report

System Design (22CS024)

BACHELOR OF ENGINEERING

in

Computer Science and Engineering

Submitted by:

Harshita-2310990601

Moorvi Garg-2310990659

Jashan Yadav-2310992105

Aditi Batra-2310992582

Group No- 24

Semester – 5th

Submitted to:

Dr. Salil Bharany

Assistant Professor



**CHITKARA UNIVERSITY, PUNJAB
CHANDIGARH-PATIALA NATIONAL HIGHWAY
RAJPURA (PATIALA) PUNJAB-140401 (INDIA)**

INDEX

Sr. No.	Contents	Page No.
1.	Problem Statement	3
2.	Goals / Objectives	4
3.	Functional Requirements and Non-Functional Requirements : Scalability, Reliability, Availability, Performance, Security, Maintainability	5 - 6
4.	High-Level Design: System Context Diagram including Major Components (API Gateway, Services, Databases, External Integrations, etc.) including the Data Flow Overview	7 - 11
5.	Detailed/Low-Level Design: Component Design/Class Diagram Data Flow - Sequence diagrams/ Use Case Diagram	12 - 14
6.	Database Design - ER Diagram or Schema	15
7.	Network System Design : Horizontal vs Vertical Scaling, Replication Strategy, CDN / Cache Layers, Recovery & Backup Strategy, Security and Privacy: Authentication & Authorization, Encryption (at rest, in transit), Access Control policies, Audit & Monitoring	16 - 18
8.	Conclusion and Future Improvements: Potential Bottlenecks	19

1. Problem Statement

Social media platforms like Instagram and Twitter depend on a dynamic news feed system to display updates, photos, and videos shared by people a user follows. The news feed is the heart of user engagement — it keeps users scrolling, interacting, and returning to the platform multiple times a day.

Designing such a system is challenging because it must handle constant activity in real time, including new posts, likes, comments, and follows. The system must not only show content instantly but also ensure content is relevant, meaningful, and personalized for each user.

The news feed must be fast, even when millions of users are online at the same time. It should smoothly manage heavy read and write operations, such as users uploading photos, liking posts, or refreshing their feed continuously. It should also be able to store and deliver large files like images and videos without delays.

In addition, the system must remain highly reliable, even if some servers fail. It needs to ensure the feed is always available, secure, and consistent across devices. Users should always see updated content, whether they are using a mobile app or a website.

Key Challenges

- Managing millions of users who post, view, and interact at the same time.
- Ranking and filtering posts so the feed feels relevant and personalized.
- Handling huge amounts of multimedia content (images, reels, videos).
- Achieving low latency — feed should load in less than 2 seconds.
- Ensuring system reliability even during server failures or traffic spikes.
- Maintaining data consistency across distributed systems and global users.

2. Goals / Objectives

The main goal of this system is to design a highly scalable, efficient, and intelligent News Feed architecture for a social media platform. The system should be capable of handling a global user base, supporting real-time content delivery, and maintaining a seamless user experience at all times.

This News Feed system should not only display posts instantly, but should also ensure that each user sees the most relevant, personalized, and meaningful content from the accounts they follow. It should operate reliably, even under heavy traffic, and continue serving content even if individual components fail.

Objectives

- To allow users to create, view, and interact with posts in real time, including text, images, videos, and stories.
- To generate a personalized news feed for each user using ranking algorithms based on recency, engagement, relevance, and user behavior patterns.
- To support real-time updates and notifications for likes, comments, shares, follows, and new posts, ensuring users feel continuously connected.
- To maintain data security and privacy through encrypted communication, secure access control, and protected data storage.
- To achieve fault tolerance and high availability through distributed architecture, load balancing, and replication across data centers.
- To ensure low latency and fast feed loading by using techniques like caching, pre-computation, and content delivery networks (CDNs).
- To make the system modular and maintainable, enabling independent scaling and easy updates to different services like feed ranking, post service, and notifications.

3. Functional and Non-Functional Requirements

Functional Requirements

- **User Account & Profile Management:**
Users must be able to create accounts, log in, log out, edit personal information, and manage their privacy and visibility settings.
- **Post Creation & Publishing:**
Users should be able to create posts with text, images, and videos, as well as edit or delete their content at any time.
- **Personalized Feed Generation:**
The feed should dynamically aggregate posts from followed users and display them in a personalized order based on recency, relevance, and engagement patterns.
- **Follow / Unfollow System:**
Users must be able to follow and unfollow others, with their feed updating automatically to include or exclude content from these users.
- **User Engagement Features:**
The system must support actions like liking, commenting, sharing, saving posts, and reacting with emojis or responses.
- **Real-Time Notifications:**
Instant alerts should be sent for likes, comments, new followers, mentions, or direct messages to enhance user interaction.
- **Search & Explore Functionality:**
Users should be able to search for people, hashtags, or trending posts using keywords, and explore recommended content.
- **Media Storage & Delivery:**
All multimedia content (images, videos, stories) should be stored securely and delivered efficiently through Content Delivery Networks (CDNs) like AWS S3 or Cloudflare.

Non-Functional Requirements

- Scalability:
The system must efficiently support millions of users and billions of daily requests through horizontal scaling, distributed architecture, and load balancing.
- Reliability:
The platform should remain operational even in case of failures by using data replication, continuous monitoring, and automated failover systems.
- Availability:
The system should achieve at least 99.9% uptime, supported by multi-region deployments and redundant servers to avoid downtime.
- Performance:
The news feed should load in under 2 seconds using caching techniques, async processing, and feed pre-computation to reduce latency.
- Security:
All data should be protected using HTTPS, JWT-based authentication, and AES encryption, ensuring users' privacy and secure communication.
- Maintainability:
Microservice-based architecture ensures easy updates, debugging, testing, and independent scaling of services like Feed, Post, and Notification.
- Consistency:
Eventual consistency must be maintained across distributed databases using asynchronous messaging queues like Kafka or RabbitMQ.
- Usability:
The interface must be simple, intuitive, responsive, and optimized for maximum user engagement across devices.

4. High-Level Design

The Social Media News Feed System follows a microservices architecture, where each service works independently and communicates through APIs or messaging queues. This enables scalability, modularity, and fault tolerance — essential for handling millions of users and posts.

Major Components:

1. Client (Web / Mobile App):

User interface for viewing feed, posting content, following users, liking posts, commenting, sharing, and receiving notifications.

2. API Gateway / Load Balancer:

Handles all incoming requests, routes them to the correct service, applies authentication, and distributes traffic evenly.

3. User Service:

Manages user registration, login, authentication, profile updates, privacy settings, followers, and following data.

4. Post Service:

Handles creation, editing, deletion, and retrieval of text, image, and video posts. Stores post details securely.

5. Feed Service:

Collects posts from followed users, ranks them by relevance and recency, and generates personalized news feeds using caching and algorithms.

6. Notification Service:

Sends real-time alerts for likes, comments, follows, mentions, and shares using asynchronous processing.

7. Cache Layer (Redis / Memcached):

Stores frequently accessed feeds, recent posts, and popular content to reduce database load and increase speed.

8. Message Queue (Kafka / RabbitMQ):

Enables asynchronous event-driven communication between services, such as new posts triggering feed updates and notifications.

9. Database Layer:

- **Relational DB (MySQL / PostgreSQL):** Stores user accounts, profiles, and follower relationships.
- **NoSQL DB (MongoDB / Cassandra):** Stores posts, feeds, and large-scale content data.
- **CDN (AWS S3 / Cloudflare):** Stores and delivers media files (images, videos) globally with low latency.

Data Flow Overview

1. Feed Request (User Opens App)

- User opens app and requests their news feed
- Request is sent to API Gateway
- API Gateway forwards request to Feed Service
- Feed Service checks Cache
 - If feed exists → returns cached feed instantly
 - If not in cache → fetches posts from Post Service
- Feed Service applies ranking algorithm (recency, relevance, engagement)
- Feed is delivered to the user and stored in cache for faster access

2. Posting New Content

- User creates a post (text, image, video)
- Request forwarded from API Gateway to Post Service
- Post metadata stored in NoSQL database
- Media stored in CDN (AWS S3 / Cloudflare)
- Post Service publishes event to Message Queue
- Feed Service updates followers' feeds
- Notification Service sends alerts to followers

3. Notifications Flow

- User action (like, comment, follow, mention) triggers an event
- Event is received by Notification Service
- Notification Service sends real-time push notifications or alerts
- Users receive updates without refreshing the app

4. Caching & Performance Handling

- Hot feeds, trending posts, and recent activity stored in Redis/Memcached
- Reduces repetitive database queries
- Ensures feed loads within 1–2 seconds

5. Media Delivery Flow

- User uploads image or video
- Media stored in CDN for global access
- Post references media URL stored in database
- CDNs ensure low-latency media delivery across regions

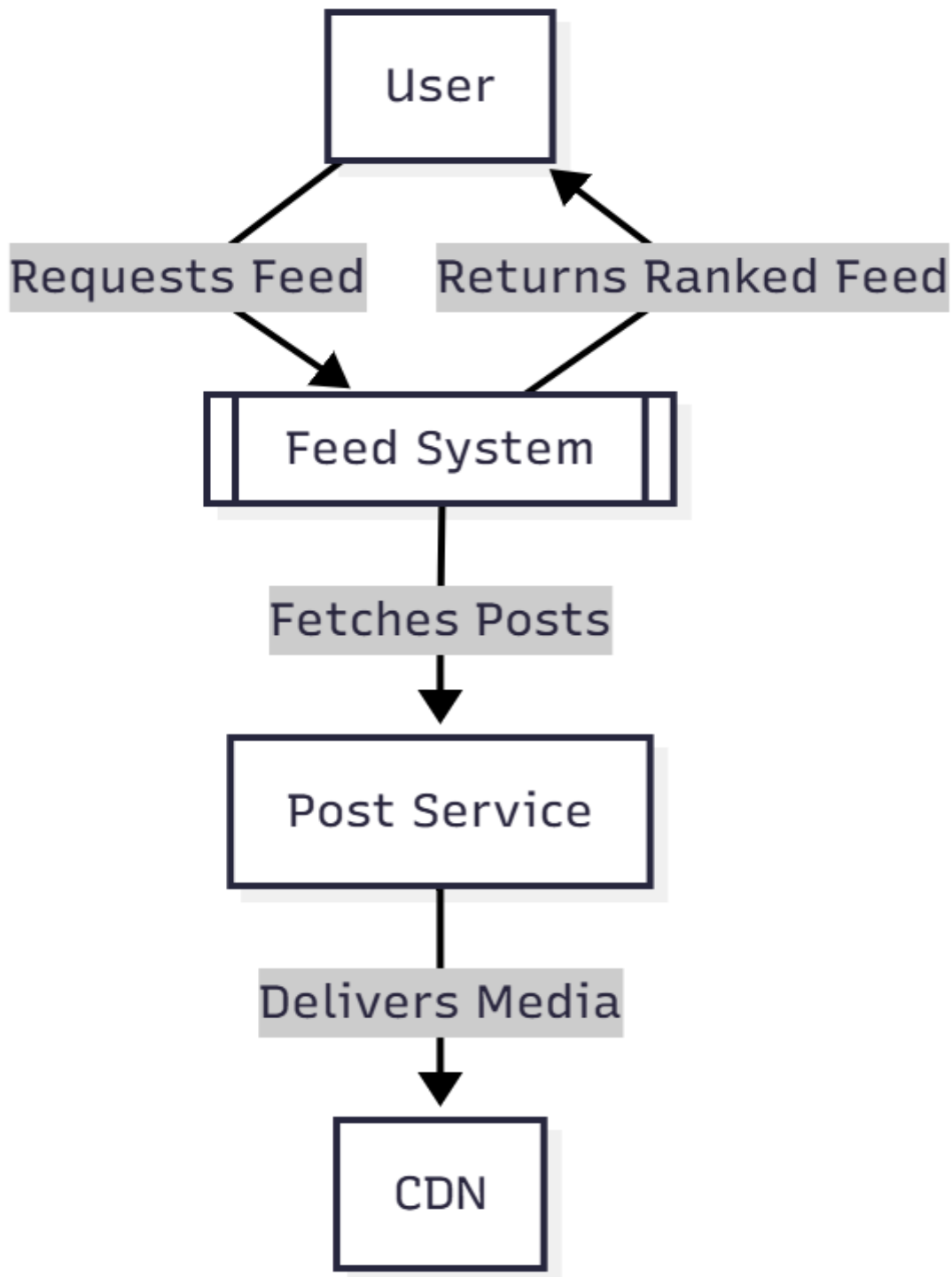


Fig 4.1 Context Diagram

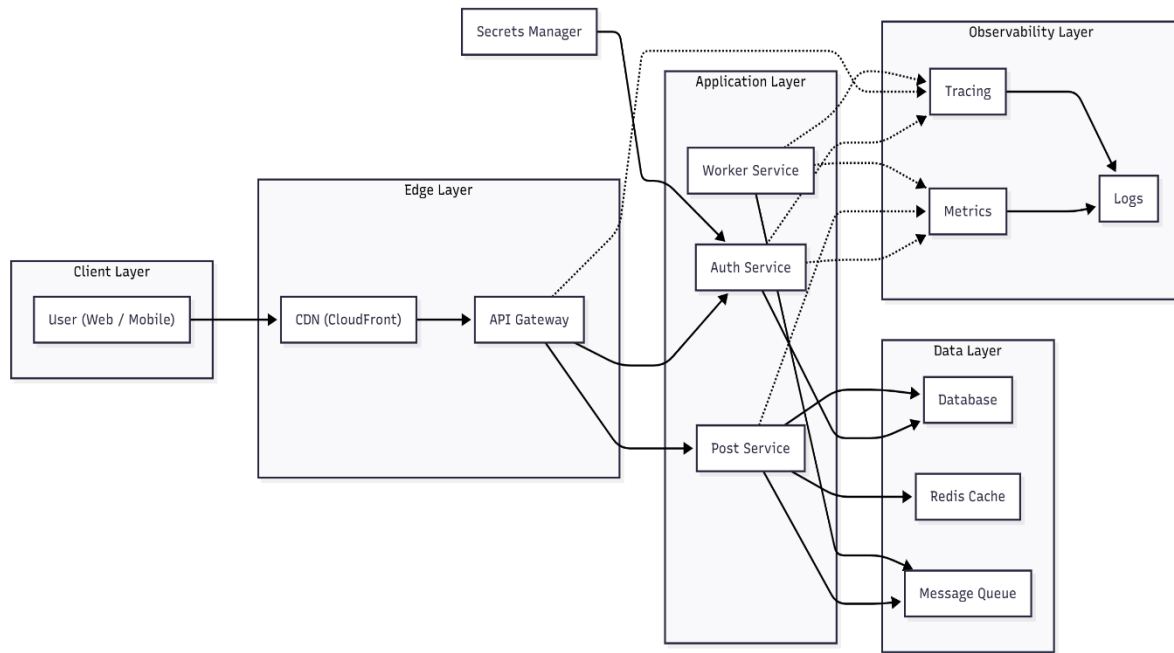


Fig 4.2 High-Level Architecture Diagram

5. Low-Level Design:

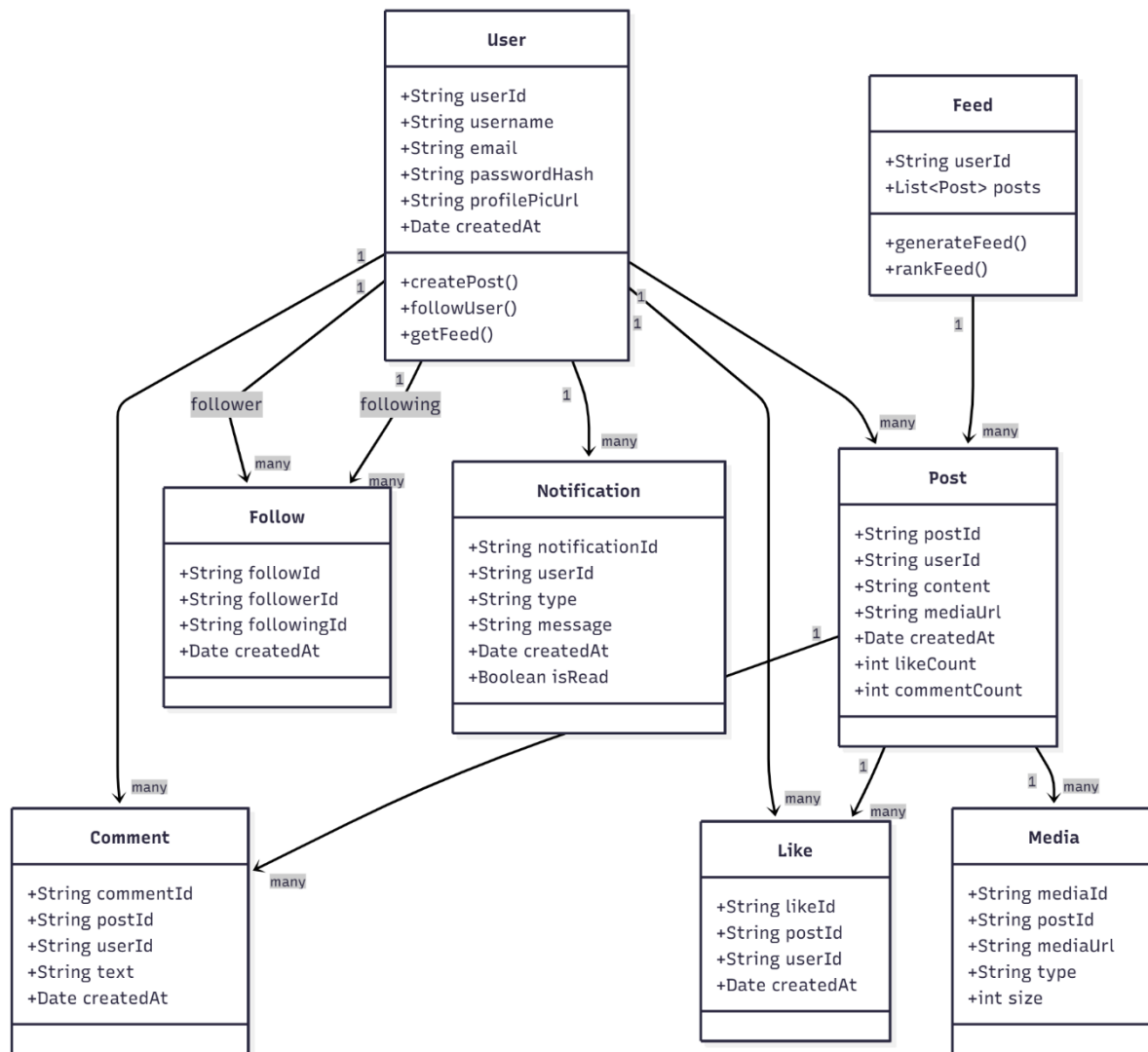


Fig 5.1 Class Diagram

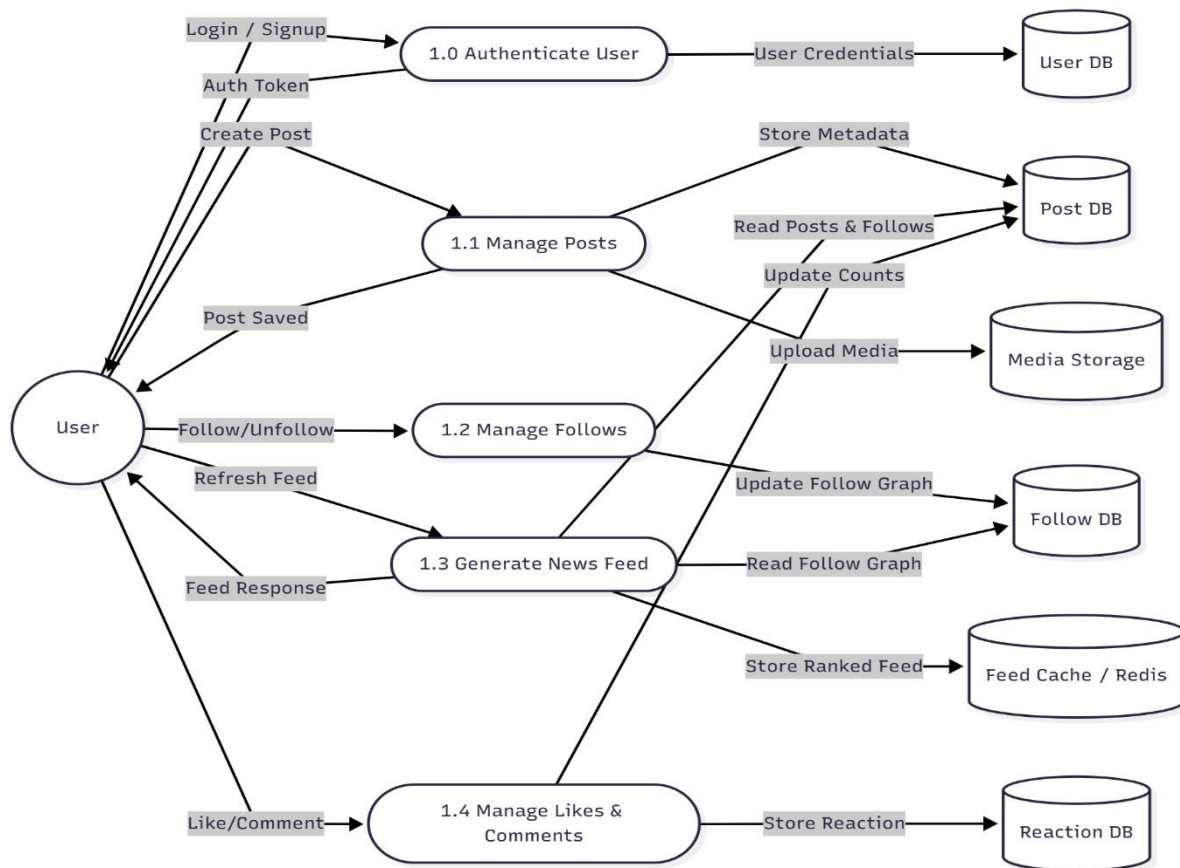


Fig 5.2 Data Flow Diagram

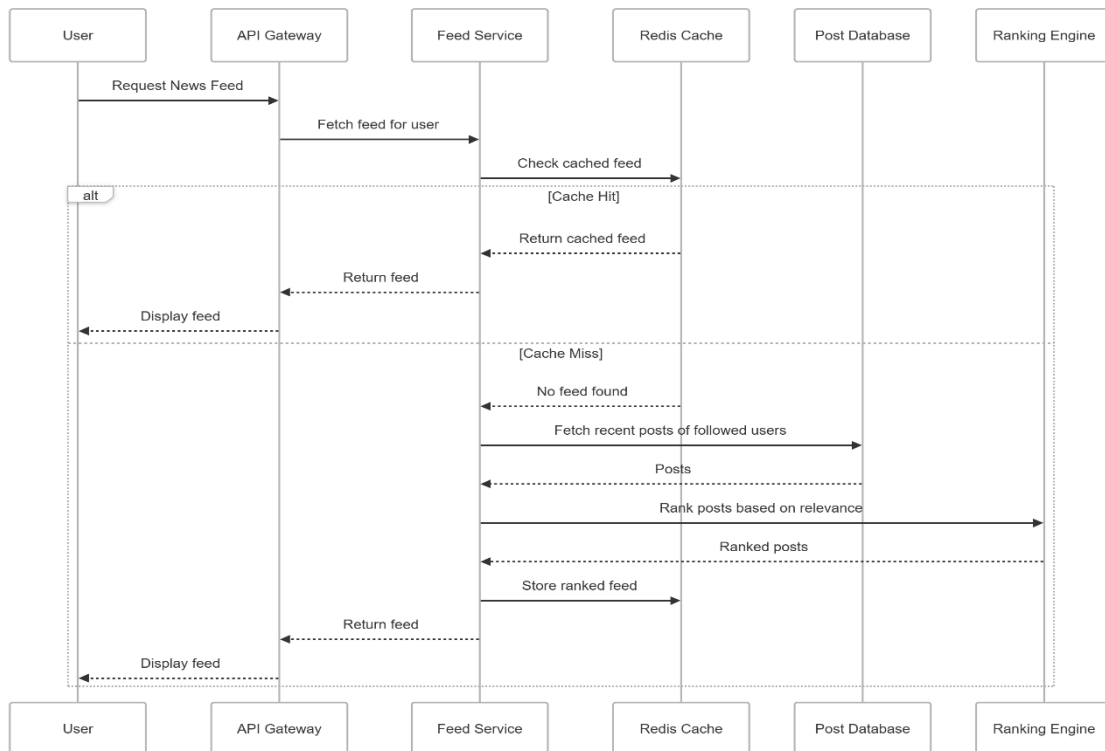


Fig 5.3 Sequence Diagram

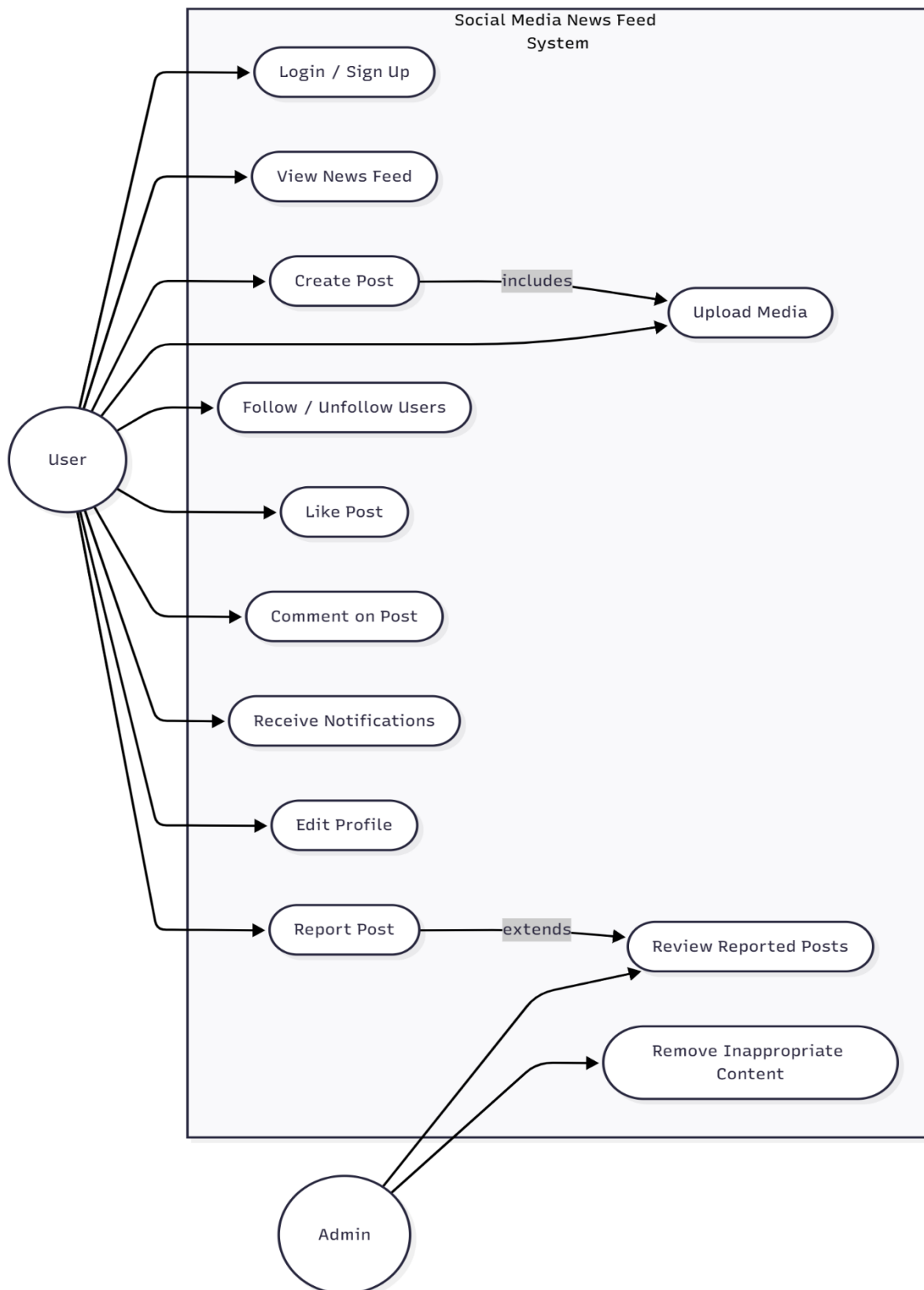


Fig 5.4 Use Case Diagram

6. Database Design- ER Diagram or Schema

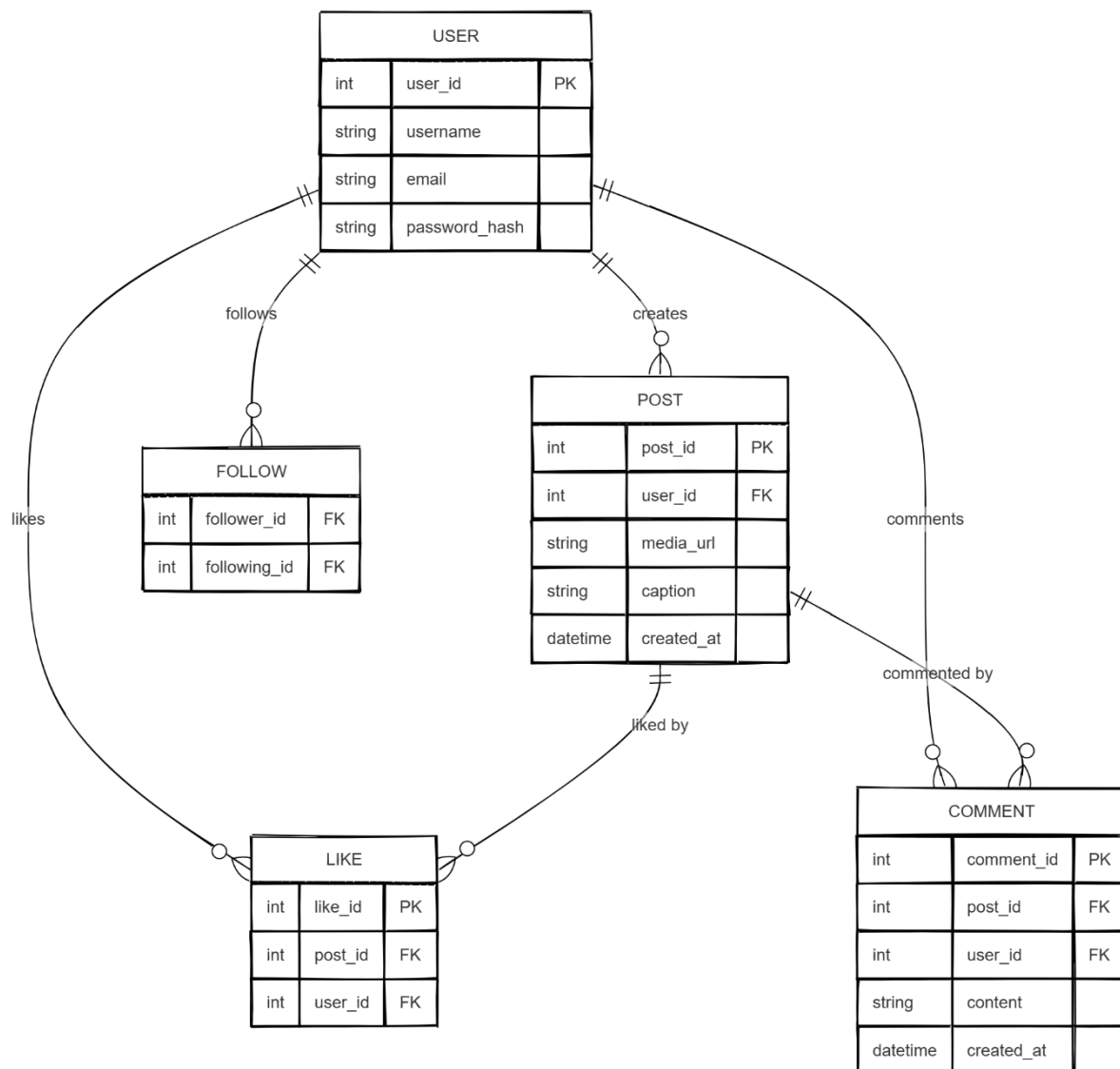


Fig 6.1 ER-Diagram

7. Network System Design

The Network System Design of the Social Media News Feed System focuses on achieving high performance, global scalability, strong security, and disaster recovery. It ensures the system remains fast, reliable, and available even during heavy load, user spikes, or server failures.

7.1 Horizontal vs Vertical Scaling

- **Vertical Scaling:** Vertical scaling means increasing the power of a single server — for example, upgrading its CPU, memory, or storage. It is useful in early stages of the platform, when user traffic is small and infrastructure is simple to manage.
- **Horizontal Scaling:** Horizontal scaling means adding more servers or nodes to handle increased load. In a news feed system with millions of users, horizontal scaling is essential for:
 - Feed Service
 - Post Service
 - Notification Service
 - Database clusters

A Load Balancer ensures traffic is distributed evenly across servers, improving responsiveness and uptime.

7.2 Replication Strategy

- **Database Replication:** The main databases are replicated across multiple servers using a Primary (Master) and multiple Secondary (Replica) nodes.
 1. The Master node handles all write operations such as posts, likes, comments, and follows.
 2. Replica nodes handle read operations such as feed loading and profile viewing. In case the Master node fails, a Replica can automatically become the new Master, ensuring zero data loss and continuous service.
- **Cache Replication:** Redis or Memcached also uses replication across multiple nodes to keep cached feed data available even if one cache node fails. This ensures quick recovery, lower latency, and uninterrupted feed delivery.
 1. This replication strategy ensures:
 2. Higher read scalability
 3. Faster access to user feeds and posts
 4. Fault tolerance and high availability

7.3 CDN / Cache Layers

- **Cache Layer:** A caching system (Redis or Memcached) stores frequently accessed feed data, trending posts, user interactions, and profile info in memory. This reduces load on the main database and ensures the feed loads within milliseconds.
- **Content Delivery Network (CDN):** The CDN is used to deliver static files (like website assets or analytics dashboards) from servers closer to the user's location. This helps reduce latency and provides faster access worldwide.

7.4 Recovery & Backup Strategy

- **Regular Backups:** Automatic database backups are taken at regular intervals (daily or weekly) and stored securely in remote cloud storage such as AWS S3 or Google Cloud Storage. Backups include user data, posts, comments, and media metadata.
- **Disaster Recovery:** If a server or database node fails, the system can restore data from the latest backup. Load Balancers redirect traffic to healthy nodes, ensuring minimal downtime.
- **Failover Mechanism:** If an entire region or data center becomes unavailable, another replica in a different geographical location takes over automatically. This allows the system to keep running without major interruption.

This strategy ensures:

- High availability
- Data integrity
- Continuity during system crashes or outages

7.5 Security and Privacy

1. Authentication & Authorization

- Only registered users can access their feed, post content, or engage with others.
- Authentication is handled through secure login methods such as OAuth 2.0 and JWT tokens.
- Authorization ensures users can only modify their own posts, profiles, and interactions.
- Admins have extended control to manage content, users, and system-wide settings.

2. Encryption (At Rest and In Transit)

- **Encryption at Rest:** All stored data — user profiles, posts, comments — is encrypted using AES-256 encryption. Sensitive data like passwords is stored using hashing (e.g., bcrypt).

- Encryption in Transit: All communication between client and server uses HTTPS (TLS/SSL) to prevent interception or data tampering.

3. Access Control Policies

- Different access levels are implemented for normal users, moderators, and system admins.
- Admins can monitor reported content, manage flagged posts, or restrict abusive users.
- Internal services communicate securely using API keys, service tokens, and firewall rules.

4. Audit & Monitoring

- Logging System: All actions login, posting, following, comments, likes are logged for security and debugging.
- Monitoring Tools: Tools like Prometheus, Grafana, and ELK Stack track performance, latency, uptime, and errors in real time.
- Alerts: Automated alerts are triggered if a service goes down, unusual spikes occur, or suspicious activity is detected.

These systems ensure:

- Transparency
- Fast issue detection
- Continuous system improvement
- Strong defence against abuse or failure

8. Conclusion and Future Improvements:

Conclusion

The Social Media News Feed System successfully fulfills the primary objectives of being scalable, secure, and highly responsive, capable of supporting millions of concurrent users while delivering real-time, personalized content. By leveraging a microservices-based architecture, the system enables modular development and flexible scaling, ensuring maintainability and future growth. The use of caching, message queues, and distributed storage significantly enhances performance, allowing feeds to load quickly even under heavy traffic. Intelligent feed ranking algorithms ensure users always receive relevant and engaging posts, while replication and failover mechanisms contribute to high availability and fault tolerance. Overall, the system delivers a seamless user experience through efficient data handling, low latency operations, and robust architecture design.

Future Improvements

- **AI-Based Feed Personalization:**
Machine learning models can be introduced to improve feed relevance using user behavior, interest patterns, and content predictions.
- **Graph Database Integration:**
Technologies like Neo4j can optimize follower and engagement relationships, enabling faster feed traversal and recommendations.
- **Edge Caching for Global Low Latency:**
Deploying feed caching at edge locations can reduce response time for international users.
- **Predictive & Auto-Scaling Load Balancers:**
Intelligent traffic management can scale system resources proactively during peak usage times.
- **Advanced Content Moderation:**
AI-driven filters to detect spam, misinformation, and harmful content in real time.